

第 4 课 · 高并发与稳定性

模型服务被打挂，才是真的挂

第 4 课 · 120 分钟（讲授 45 + 实操 65 + 小面试 10）

一、这节课的目标

三件事：第一，理解 agent 系统在并发下的**真实失败模式**；第二，搞清楚四道限流和一道闸门分别在保护什么；第三，自己压一把，看着系统降级、再自己恢复。

这是六节课里最「工程」的一节，也是面试得分点最集中的一节。代码在 `src/ailab_ops/serving/`，六个模块。

二、口语化讲稿

先讲清楚：agent 系统为什么和普通 Web 服务不一样

普通后端服务的压力模型是：请求很快、QPS 很高、数据库是瓶颈。agent 系统完全不是这样，有六个本质区别。

区别	后果
请求时长从毫秒级变成秒/分钟级	超时策略、连接占用完全不同
流式 + 长连接	thread-per-request 模型直接死
有状态（多轮会话、工具中间态）	水平扩展时要做会话亲和或共享存储

区别	后果
上游（GPU 推理）是关键稀缺资源	**瓶颈几乎永远不在 API 层**
每个 token 花钱	需要计量和预算，不只是限速
超时不等于失败	可能已经生成完；重试有副作用的工具会重复执行

第四行是这整节课的核心。你的瓶颈**几乎永远不是**你的 API 服务，而是后面那个模型服务。所以整套并发设计的核心任务不是「让 API 扛住」，而是「**保护上游，并且公平地分配上游**」。

模型的失败模式：不是变慢，是全挂

这一点我想讲透，因为它决定了后面所有设计。一个典型的推理服务同时只持有**少量序列**——可能几个到几十个。超过这个数之后，延迟超线性上升，然后开始拒绝、重启、或者 KV cache 疯狂抖动。

所以模型服务被压垮时的表现，**不是「请求变慢了」，而是「每一个请求都失败了」**——包括那些本来能成功的。这个区别要命：变慢只是体验差，全挂是事故。

这就是为什么必须有闸门：它的作用不是提速，是**让系统在过载时只失败一部分**，而不是全部。

四道限流 + 一道闸门，各管什么

很多人把这五样东西混成一个「限流」，这是错的。它们回答的是五个不同的问题，少了任何一个都会以不同的方式出事故。

机制	回答什么问题	没有它会怎样
单用户 QPS	是不是某个调用方在滥用？	一个客户端拖垮所有人
租户并发	一个组织的模型用量占了多少？	一个吵闹的租户打满整个集群
租户 token/分钟	持续生成量是多少？	一直无法缓解的过载
租户美元/天	这要花我们多少钱？	一张意外的账单
上游闸门	允许多少个调用同时在飞？	**模型服务被打挂**

租户这一层容易忽略。真正会出事的不是单个用户在滥用，而是**一个组织里一百个用户在同时用**。而为此负责的是那个组织，不是某一个人。所以限流必须按租户也做一遍。

而且输出会**明确标注**这是降级结果。一个没标注的低质量答案，比一个标注了的更危险——工程师需要知道这个结论能不能信。

熔断器：别把故障放大

- 连续失败 N 次后打开，冷却期内直接拒绝，冷却后放一个探针
- **按租户**分开熔断：一个租户的畸形请求不该熔断所有人
- 打开了就要降级，而不是继续往一个已经挂了的服务上打
- 重试一定要带抖动（jitter）——否则所有客户端同时重试，会把一次上游抖动放大成持续故障

最后一条叫「重试风暴」，是自己给自己制造的雪崩。

三、动手：压一把

实操 1：让闸门真正咬住

必须先加 --llm-latency-ms。模型瞬时返回的话，永远只有一个调用在飞，闸门根本不会吃紧。

```
PYTHONPATH=src python3 -m ailab_ops.cli serve --llm-latency-ms 400    # 00 1
make bench                                                            # 00 2
```

这个 `--llm-latency-ms` 是个教学开关，但它揭示了一个真实问题：**如果模型很快**，你的并发控制就永远不会被测到**，而你不知道它是好是坏**。真实系统里模型就是慢的，所以真实系统里闸门一定会吃紧。

实操 2：读懂三个场景的报告

场景	打开哪个机制	该看到什么
steady	全部天花板抬起	120/120 成功、零拒绝。 这里出现 429 就是 bug

场景	打开哪个机制	该看到什么
burst	只有闸门 (缩到 4 槽队列)	大量 queue-full 拒绝, in_flight 稳定在上限
hotkey	只有缓存	大部分走 cache, gate.admitted 几乎不动

注意 burst 场景的设计：它**故意把闸门缩到很小**。因为队列如果足够大，整个 burst 都能排队成功，那就什么也没演示到——只是慢一点而已。**准入控制只有在队列真的小于负载时才可见**。这一点在真实系统里同样成立。

实操 3：注入故障，看系统自愈

降级不是失败。注意 served_by 字段变成了 evidence_only。

```
# Web UI kill the model service
# 
curl -X POST localhost:8080/v1/admin/force-degrade?on=true
#  — degraded
curl -X POST localhost:8080/v1/diagnose \
  -H 'Content-Type: application/json' \
  -d '{"question": "job-xxx"}'
```

观察两件事：第一，答案里 `served\_by` 变成了 `evidence\_only`，而且 degraded_reason 说明了为什么降级。第二，恢复模型之后，下一次请求自动回到 `full` ——不需要人工干预。

四、小面试（10 分钟）

Q1：大模型服务被判挂了，你怎么排查？

这是本课最典型的面试题。按这个顺序答：

1. **先分层**：是客户端问题、网关问题、还是推理服务本身？看错误类型——连接超时、读超时、5xx、429，指向不同层。
2. **看是不是雪崩**：如果所有请求都挂，先怀疑重试风暴。把重试关掉看是否恢复。
3. **看上游容量**：同时在飞的调用数是多少？超过推理服务承载能力了吗？
4. **看有没有降级**：熔断器打开了吗？降级阶梯走到哪一级了？
5. **最后才看模型本身**：是不是某个版本的模型有问题。

这个顺序体现的是**从系统到个体**的排查思路。一上来就说「我看看模型输出对不对」的人，会被认为没做过生产。

Q2：为什么要区分「租户并发」和「单用户 QPS」？

因为滥用的形态不同。单用户 QPS 防的是**一个脚本**在猛刷；租户并发防的是**一百个正常用户**同时用。后者看起来每个用户都很规矩，但合起来能把集群打满，而且责任主体是那个组织。只做单用户限流的话，这种情况完全拦不住。

Q3：为什么成本要用「每天」而不是「每分钟」限？

因为按分钟限可以被规避——把同样的开销摊到一个小时里，每分钟都不超限，但总额照样爆。**总量限制必须用长周期**。

Q4：超时和重试，怎么设计才不出事？

- 总截止时间要小于客户端超时，否则客户端先放弃，你的工作白做
- 重试必须区分可重试与不可重试：429/5xx 可重试；400/401/403 重试是浪费
- **硬配额（quota）不可重试**——重试不创造配额，只会把窗口烧光
- 重试一定要带指数退避 + 抖动
- 有副作用的操作必须幂等才能重试

本项目里 llm/openai_compat.py 的 `classify\_status` 就是干这个的，它把 429 分成「限流」和「配额」两类，前者可重试后者不可。

五、作业

1. 把 `AILAB\_UPSTREAM\_CONCURRENCY` 分别设成 2、8、32，各跑一次 burst，记录拒绝率和 p99 延迟，画出曲线。想清楚为什么并发不是越大越好。

2. 把 `AILAB\_TENANT\_COST\_PER\_DAY\_USD` 设成 0.01，跑几次诊断，观察 `cost_exceeded` 的拒绝。特别注意：**这个拒绝不会因为重试而解除**。对比一下 429 限流，体会两者的区别。
3. 思考题：如果让你给这套系统加一个「VIP 租户」机制，你会改哪里？提示：看 `gate.py` 里的 `Priority` 和 ``try_promote_starved``。